

Developer Documentation

≡ Synera Headless

Introduction

Synera Headless is a console application that allows you to run Synera without the graphical user interface (GUI) in background. It supports several commands that are listed below.

Setup

If you are a developer, it's recommended to add reference to `SyneraHeadlessModel.dll` in your project. `SyneraHeadlessModel` contains models that can be used for serialization/deserialization of inputs/results of `syneraheadless.exe`. This library is a .NET Standard 2.0 project and doesn't have any additional dependencies, so you shouldn't have any problems.

Commands

Run `syneraheadless.exe --help` to see all commands.

Exit code of the application is `0` if the run was successful and `1` if at least one error has occurred. Console log and `--output t` file contain the detailed information about the errors.

info

Shows information about the specified file.

Example that shows information about `file.syn` and writes the result to `output.json`:

```
syneraheadless.exe info --file file.syn --output output.json
```

execute

Executes the specified workflow by injecting inputs passed as command line arguments. The workflow must have inputs and/or output defined.

Example that executes `file.syn` with parameters `5`, `import.stl` `export.step` and writes the result to `output.json`:

```
syneraheadless.exe execute --file file.syn --input 5 import.stl export.step --output output.json
```

Similar example that loads inputs from `input.json`:

```
syneraheadless.exe execute --file file.syn --jsoninput input.json --output output.json
```

upgrade

Upgrades the specified file or a group of files to the most recent format. This command allows you to batch update all .els files to .syn format.

Example that upgrades file.syn:

```
syneraheadless.exe upgrade --file file.syn
```

Example that upgrades file.els. As a result the command a new file file.syn is created.

```
syneraheadless.exe upgrade --file file.els
```

Example that upgrades all files in C:\Synera Documents\

```
syneraheadless.exe upgrade --directory "C:\Synera Documents"
```

Example that upgrades all files in C:\Synera Documents\ that contain word FEM in the name.

```
syneraheadless.exe upgrade --directory "C:\Synera Documents" --searchpattern *FEM*
```

removeTestNodes

Removes the test nodes of the workflows automatically. It can be called together with the --file command to be executed only for a specific file path. Use the --files option that supports multiple files separated by a space.

Example that removes test nodes from file.syn:

```
syneraheadless.exe removeTestNodes --file file.syn
```

Example that removes test nodes from multiple file.syn:

```
syneraheadless.exe removeTestNodes --files file1.syn file2.syn
```

The --interactive mode allows you to specify any number of file paths at runtime.

```
C:\Program Files\Synera>SyneraHeadless.exe removeTestNodes --interactive
Enter file path (enter empty line to finish):
D:\TestNode.syn
Removing test Nodes from D:\TestNode.syn...
```

Example that removes test nodes from files within a given directory e.g. C:\Synera Documents\.

```
syneraheadless.exe removeTestNodes --directory "C:\Synera Documents"
```

Example that removes test nodes from files that contain a specific word e.g. example in the name within a given directory e.g. C:\Synera Documents\.

```
syneraheadless.exe removeTestNodes --directory "C:\Synera Documents" --searchpattern *example*
```

Options

To see all options run syneraheadless.exe command --help. For example, syneraheadless.exe execute --help.

--output

Writes the result of execution to the specified file. To programmatically read the output, deserialize it to `Synera.Headless.Model.ExecuteResult` or to `Synera.Headless.Model.InfoResult` class.

Example that reads and deserializes the output file:

```
var json = System.IO.File.ReadAllText(path);
var settings = new JsonSerializerSettings { Error = (s, e) => { e.ErrorContext.Handled = true; } };
var result = JsonConvert.DeserializeObject<ExecuteResult>(json, settings);
```

--input

Let's assume that our file has 4 inputs

- `x`, Number
- `Flag`, Bool, optional
- `File`, String, default value `output.stl`
- `Center`, Point

Then you can specify the inputs like this:

- Specify all inputs (order matters): `--input 3 true "My Folder\My File.stl" 10,0,5`
- Skip optional input(s) with a special value `""`: `--input 3 "" "" 0,0,0, --input 3 true "" 0,0,0, --input 3 "" "Hello" 0,0,0`
- Completely skip all optional inputs: `--input 3 1,0,0`

Hints:

- You can specify any type as long as it can be parsed from a string
- Empty string `""` means that the value is skipped
- You can use quotes `"value with spaces"` and `\` as usual (e.g. for paths)
- If you want to use complex type, list/tree inputs or struggle with escaping, use `--jsoninput`

Automatic import

It is possible to automatically import files without using dedicated Import nodes. Simply specify the file path as an input and the file will be automatically imported (the type of the input should be compatible with the file format). For example, if the file has 3 inputs of types `Number`, `Body` and `Mesh`, the following command allows to import the geometry files from disk: `--input 5 "my body.stp" "my mesh.stl"`.

--jsoninput

Specifies that inputs should be read from the specified JSON file.

```
syneraheadless.exe execute --file file.syn --jsoninput input.json
```

If you are a user, you can use `--template` option to generate template JSON file that you can edit manually. See `--template` for more details.

If you are a developer, you can generate input file programmatically:

```
var inputs = new List<WorkflowInput>
{
    // single value
    new WorkflowInput
    {
        Id = someId,
        // Name = "Radius" // you need to specify either Id or Name; if you specify both, Name is ignored
        Value = new DataTree
        {
            StringValue = new Dictionary<string, List<string>>
            {
                ["0"] = new List<string> { "<Enter value here>" }
            }
        }
    },
    // list of values
    new WorkflowInput
    {
        Id = anotherId,
        Value = new DataTree
        {
            StringValue = new Dictionary<string, List<string>>
            {
                ["0"] = new List<string> { "1", "2", "3", "4" }
            }
        }
    },
    // tree of values
    new WorkflowInput
    {
        Id = yetAnotherId,
        Value = new DataTree
        {
            StringValue = new Dictionary<string, List<string>>
            {
                ["0"] = new List<string> { "1", "2", "3", "4" },
                ["1"] = new List<string> { "10", "20" },
                ["1,1,0"] = new List<string> { "25", "100" },
            }
        }
    },
};
var json = JsonConvert.SerializeObject(inputs);
System.IO.File.WriteAllText(path, json);
```

This way you don't need to reference Synera SDK, however you're limited to values that can be converted from a string e.g. Number, Boolean, Point, Vector, etc.

To use any canvas type, you need to reference Synera SDK:

```
var inputs = new List<WorkflowInput>
{
    new WorkflowInput
    {
        Id = someId,
        Value = new DataTree
        {
            Value = new Dictionary<string, List<object>>
            {
                // any Synera type that implements IGraphDataType
                ["0"] = new List<object> { body, mesh, voxel }
            }
        }
    },
    new WorkflowInput
    {
        Id = anotherId,
        Value = new DataTree
        {
            Value = new Dictionary<string, List<object>>
            {
                // You must use SyneraDouble, SyneraBool, SyneraString
                ["0"] = new List<object> { new SyneraDouble(1), new SyneraDouble(2), new SyneraDouble(3) }
            }
        }
    },
};

var serializer = new SyneraJsonSerializer(); // Or Application.Current.Serializer
var writer = serializer.GetWriter();
writer.Write(inputs);
using (var stream = System.IO.File.Create(path))
{
    writer.Save(stream);
}
```

--exportformat, --exportdirectory & --exportnaming

The options allow to export the outputs in the given format. The format is given in a form of a file extension for each output; if an output shouldn't be exported, an empty string "" should be specified. In the example below, the 1st output is exported as an .stl file, the 3rd output is exported as an .x_t file and the 2nd output is not exported at all.

```
syneraheadless.exe execute --file file.syn --input 5 --exportformat .stl "" .x_t
```

The export format should match the type of the respective output (e.g., in the example above the 1st output should be a mesh and the 3rd output should be a Body).

--exportdirectory specifies the directory to which the data will be exported. By default the data is exported to {env file location}/Results.

--exportnaming defines the naming schema of the exported files:

- `--exportnaming Name` (default) - the names of the files match the names of the outputs; this could lead to collisions if the output names are not unique
- `--exportnaming Guid` - the names of the files match the GUIDs of the outputs

--template

Generates template JSON file for `--jsoninput`.

Example that creates template `template.json` for `file.syn`:

```
syneraheadless.exe info --file file.syn --template template.json
```

Template looks like this:

```
[
  {
    "id": "a9748744-e8be-4f5f-9170-ae5ea9dc262",
    "name": "Radius", // If id is set, this field is not required. It was added for convenience
    "typeName": "Floating point number", // This field is not required. It was added for convenience
    "value": {
      "stringValue": {
        // 0 is path in the tree
        "0": [
          "<Enter value here>" // you can enter any value that can be converted from a string, e.g. Number, Boolean, Poi
        ]
      }
    }
  },
  {
    "id": "8ebe4a63-f45a-4afc-8c5c-92aa600c631a",
    "name": "Tolerance",
    "typeName": "Floating point number"
    // This input is optional or has default value, so the value field is omitted. You can add it manually
  }
]
```

To create a list of values, do the following (here we create a list with 3 items):

```
[
  {
    "id": "a9748744-e8be-4f5f-9170-ae5ea9dc262",
    "value": {
      "stringValue": {
        "0": [
          "Hello World",
          "True",
          "25"
        ]
      }
    }
  }
]
```

```
}
]
```

To create a tree, do the following (here we create a tree with 3 branches: 0, 1 and 1,1,0):

```
[
{
  "id": "a9748744-e8be-4f5f-9170-ae5ea9dc262",
  "value": {
    "stringValue": {
      "0": [
        "Value 1",
        "Value 2",
        "Value 3"
      ],
      "1": [
        "Value 5"
      ],
      "1,1,0": [
        "Value 10"
      ]
    }
  }
}
]
```

--canvasscreenshot & --canvasscreenshotquality

Allows to generate screenshot of the canvas. You can adjust quality of the image with `--canvasscreenshotquality` option (default is 1). E.g. `--canvasscreenshotquality 4` would generate a screenshot that is 4 times bigger than the default one and corresponds to 400% zoom level on canvas. Due to memory limitations, it might not be possible to generate screenshot with the specified quality; in such case quality is automatically decreased.

Example that executes `file.syn` with parameters 5, 10 and generates a screenshot after execution is finished:

```
syneraheadless.exe execute --file file.syn --input 5 10 --canvasscreenshot canvas.png
```

Similar example with higher screenshot resolution:

```
syneraheadless.exe execute --file file.syn --input 5 10 --canvasscreenshot canvas.png --canvasscreenshotquality 2
```

--allownulls

By default execution is considered a failure if an input or an output yields an empty tree or `null` value. This could happen if data conversion fails (e.g. input expects a number, however I'm a string, not a number was passed instead) or when upstream node outputs `null` due to execution error. `--allownulls` overrides this behavior and solution is considered successful no matter what.

Execution will not run, because string `not a number` cannot be converted to a number and the application will exit prematurely with exit code 1:

```
syneraheadless.exe execute --file file.syn --input 5 "not a number"
```

If you add `--allownulls`, second input would yield `null` due to conversion error, however solution will continue. After solution is finished, application exits with error code 0:

```
syneraheadless.exe execute --file file.syn --input 5 "not a number" --allownulls
```

Has this article been helpful?



[Imprint](#)

[Data Privacy](#)

[Terms](#)

© 2026 Synera GmbH. All Rights Reserved.